

Exercice 1 — JDBC : connexion et requêtes sécurisées

Thème : *DriverManager*, *DataSource*, *Statement* vs *PreparedStatement*

Contexte

On dispose d'une table SQL **etudiants** avec les colonnes **id** (INTEGER, clé primaire), **nom** (VARCHAR) et **email** (VARCHAR).

Objectifs

- Établir une connexion JDBC à une base de données.
- Insérer un nouvel étudiant à partir de valeurs saisies par l'utilisateur.
- Rechercher un étudiant par son nom, saisi par l'utilisateur.
- Expliquer pourquoi l'approche choisie protège contre les injections SQL.

Travail demandé

- 1 Écrire une méthode `getConnection()` qui obtient une connexion via **DriverManager** à partir d'une URL JDBC.
- 2 Écrire une méthode `ajouterEtudiant(String nom, String email)` qui insère une ligne dans la table **etudiants**. Cette méthode doit utiliser un **PreparedStatement**, jamais une concaténation de chaînes.
- 3 Écrire une méthode `rechercherParNom(String nom)` qui retourne l'étudiant correspondant, en utilisant également un **PreparedStatement**.
- 4 Dans un commentaire, donnez un exemple de valeur de nom qui, avec un **Statement** concaténé, provoquerait une injection SQL, et expliquez pourquoi le **PreparedStatement** neutralise ce risque.
- 5 Fermez correctement les ressources JDBC (`Connection`, `PreparedStatement`, `ResultSet`), de préférence avec un bloc `try-with-resources`.

Critères d'évaluation

- Aucune requête SQL n'est construite par concaténation de variables.
- Les ressources JDBC sont systématiquement fermées.
- L'explication sur l'injection SQL est correcte et précise.